

TP5- Filtrage RII en temps réel

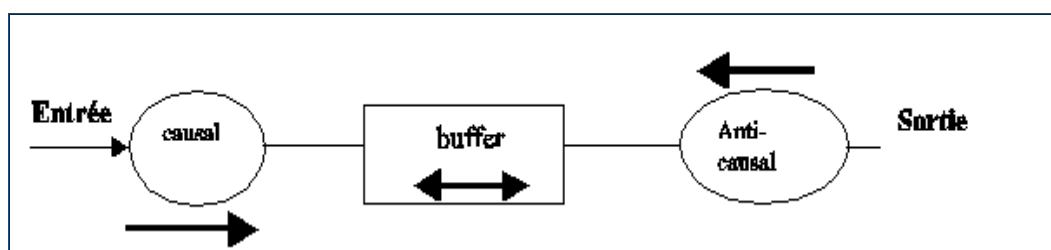
Filtre de Deriche 1D (asm & C)

Le filtre de Deriche 1D

Le filtre de Deriche est un filtre récursif d'ordre 2. Il peut être réalisé en cascade de cellules de base (filtre récursif d'ordre 1). L'équation de la cellule de base est :

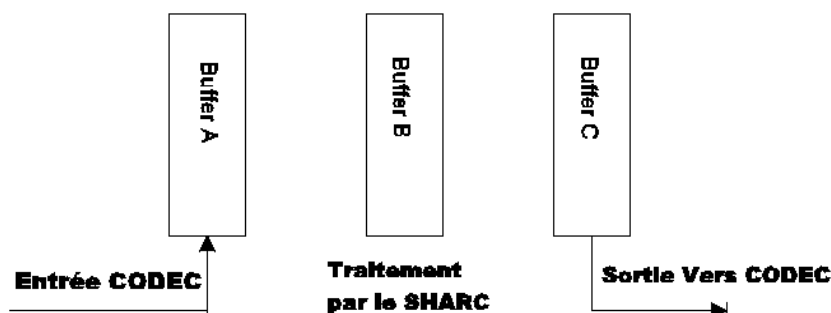
$$Y_n = (1 - \gamma) * X_n + \gamma * Y_{n-1}$$

Le filtre 1D est constitué d'une partie causale et d'une partie anti-causale. Chacune de 2 parties est réalisée en cascade de 2 cellules de base. La partie causale consiste à appliquer le traitement sur les données du flux d'entrée. Le résultat est alors stocké dans un buffer de longueur N. La partie anti-causale consiste à appliquer le même traitement au contenu du buffer précédent mais balayé dans le sens contraire de son remplissage par la partie causale. Après avoir traité les N échantillons le cycle recommence avec N nouveaux échantillons.



Implantation du filtre de Deriche 1D sur EZ-Kit Lite

L'objectif de cette partie est d'utiliser le module EZ-Kit Lite pour filtrer un signal en appliquant le filtre de Deriche. Pour ce faire, on va utiliser l'entrée analogique connectée au CODEC. Le SHARC lit les échantillons du CODEC et lui renvoie les résultats du traitement à une fréquence fixe (programmable). La fréquence des échanges entre le SHARC et le CODEC d'une part et la fréquence de traitement des données d'autre part sont différentes. Pour implanter cet algorithme, il est donc indispensable d'utiliser 3 buffers.



Un buffer sera réservé pour stocker les échantillons envoyés par le CODEC. Un autre buffer contiendra les données précédemment lues sur lesquelles le traitement sera appliqué et enfin un troisième pour contenir les résultats du traitement précédent (à sortir vers le CODEC).

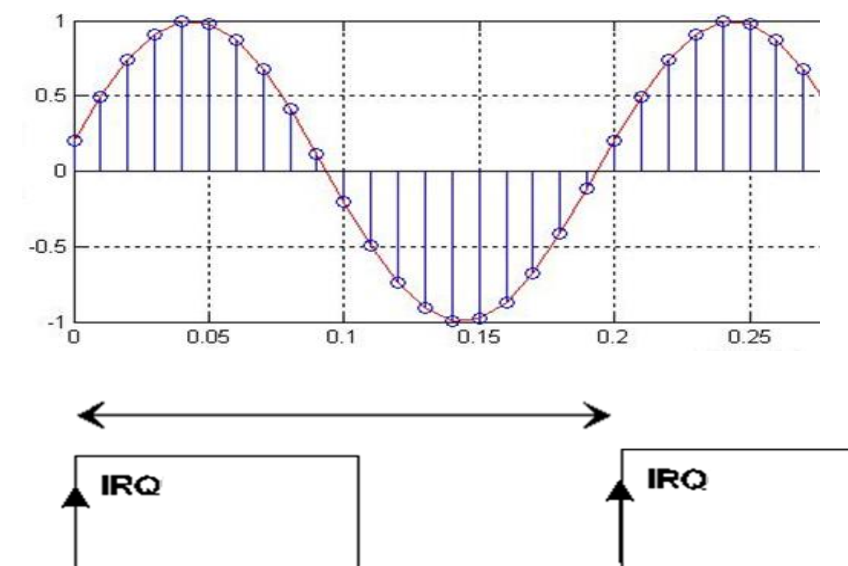
Il s'agit donc de mettre en place un pipeline à 3 étages permettant de désynchroniser les phases d'acquisition/restitution (CODEC) et le traitement (DSP) :

1. Pendant la première phase, le CODEC remplit le buffer A pendant que le traitement se fait sur les données du buffer B. Le buffer C est vidé vers le CODEC.

2. Dans la phase 2, le CODEC utilise le buffer C pour stocker les nouveaux échantillons, et récupère le résultat du traitement précédent contenu dans le buffer B. Le SHARC exécute le traitement sur les données du buffer A.
3. Durant la dernière phase, le SHARC traite le buffer C pendant que le buffer A est envoyé au CODEC. Ce dernier stocke les nouvelles valeurs dans le buffer B.

Le cycle recommence pour l'acquisition d'un nouveau bloc de données.

Comme tout filtre récursif, le lisseur de *Deriche* effectue un traitement sur un **bloc** de données. Un signal de synchronisation est donc indispensable pour annoncer le début de chaque bloc (et donc la fin du précédent). Ce signal de synchronisation va rythmer le pipeline à 3 phases : la fin d'une phase et le passage à la phase suivante. Pour valider l'implantation du filtre, nous utiliserons, comme signal de synchronisation, le signal TTL issu du générateur de signaux périodiques (même fréquence). Pour cette réalisation, un bloc de données est représenté par les échantillons d'une période complète du signal.



Nous allons exploiter le front montant du signal carré TTL pour passer d'une phase à une autre. Pour ce faire, nous associerons une interruption, nommée "**IRQ_P1**", à la broche **DAI_P1** de l'**Interface Audio Numérique** du DSP, qui est câblée mais non utilisée sur le kit. Le front montant du signal TTL servira à provoquer une interruption matérielle pour signaler le début d'un bloc d'échantillons.

L'interface DAI (pour *Digital Audio Interface*), nouvelle fonctionnalité des processeurs SHARC, est un ensemble de périphériques comprenant un contrôleur d'interruptions audio, un port de données et une unité de routage des signaux (SRU) permettant un routage logiciel des ports d'entrées/sorties du DSP.

Les entrées d'interruptions pour le DAI peuvent être : MISCA0 à MISCA3 et MISCB0 à MISCB5

Les registres à utiliser pour configurer les interruptions du DAI sont :

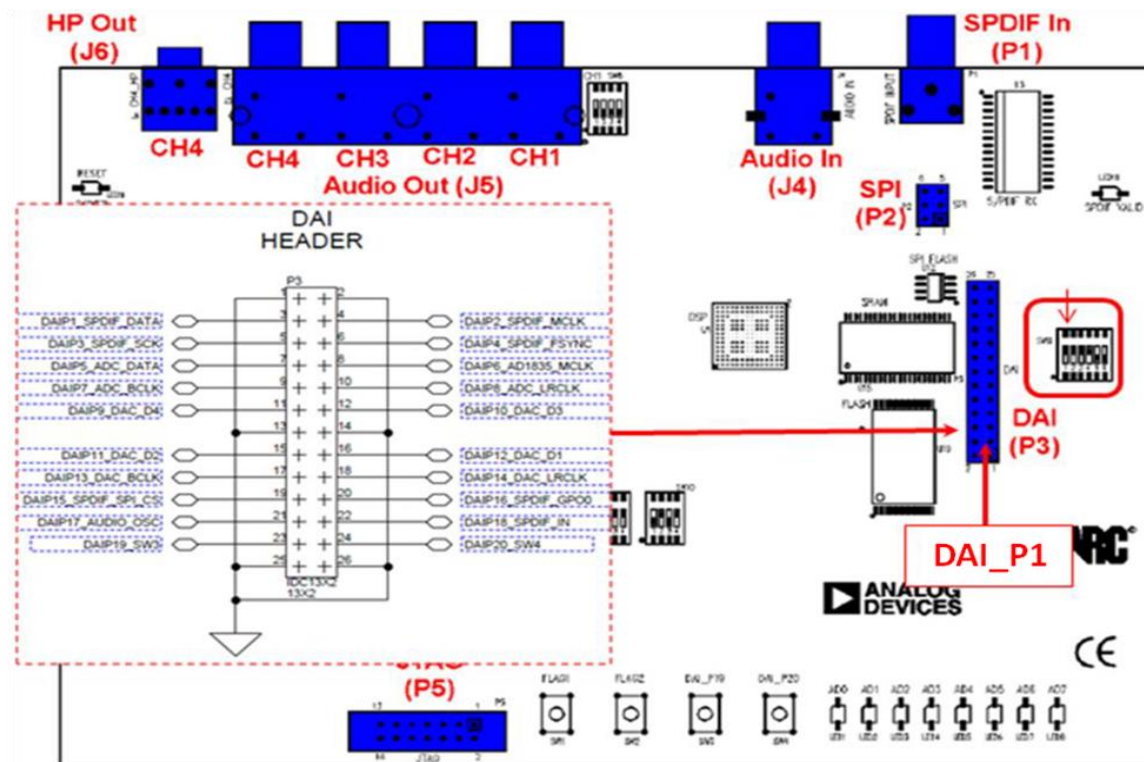
DAI_IRPTL_RE : pour valider l'interruption sur front montant
DAI_IRPTL_FE : pour valider l'interruption sur front descendant
DAI_IRPTL_PRI : pour attribuer des niveaux de priorité

Voici le code à écrire dans **main** (ne pas oublier d'inclure **sru.h**) :

```
*pDAI_IRPTL_PRI = SRU_EXTMISCB1_INT; // Démasquer individuellement l'interruption
*pDAI_IRPTL_RE = SRU_EXTMISCB1_INT; // Choisir le front montant pour l'interruption

// Unité de routage : Router (DAI_P1 ----> MISCB1)
SRU(LOW,DAI_PB01_I);
SRU(DAI_PB01_O,MISCB1_I);
SRU(LOW,PBEN01_I);
```

☒ **Une dernière chose à faire : vérifier que: le switch 2 de SW8 est bien positionné sur "off".**



Travail demandé :

Pour réaliser l'implantation du filtrage temps réel, il est fortement recommandé de suivre les étapes ci-dessous. N'oubliez pas de valider les étapes par le professeur présent.

1. Nous allons tester le fonctionnement de l'interruption **IRQ_P1**. Pour ce faire, utiliser un générateur de signaux et connecter la sortie TTL sur l'entrée **IRQ_P1** (ne pas oublier la masse GND). En s'inspirant des exemples précédents sur les interruptions, écrire une fonction de déroutement pour l'interruption **IRQ_P1** pour faire clignoter les 8 LEDs en fonction de la fréquence du signal analogique.

```
//      IRQ_P1 - Interrupt runtime
// -----
void my_irqP1( int sig_num )
{
    static int IT_register ;

    IT_register = *pDAI_IRPTL_H;
    sig_num = sig_num ;
    if ((IT_register & SRU_EXTMISC1_INT) != 0) // vérifier que l'origine de l'IT de DAI est DAI_P1
    {
        ..... // à compléter pour faire clignoter les 8 LEDs
    }
}
```

L'interface DAI possède deux interruptions programmables :

- **SIG_DAIH** : interruption avec une priorité élevée (*higher priority option*)
- **SIG_DAIL** : interruption avec une priorité basse (*lower priority option*)

➤ Créer un projet pour ce travail et valider le fonctionnement de l'interruption DAI_P1.

2. **Validation du filtre en simulation** (sans E/S, à faire dans le projet de TP2 : **TP2_rii**)
 - Ecrire en C la fonction du filtre de *Deriche*. Le prototype de la fonction doit être choisi comme suit : **void Lisseur1D (float *tab, float gamma, int nsamples)**.
Nommer le fichier correspondant "*Lisseur1D.c*"
 - Simuler la fonction **Lisseur1D** avec les paramètres suivant :
 - Coefficient *gamma* = 0.5
 - Utiliser le signal du tableau "carre_100b.dat"

Question : Donner les résultats de simulation pour trois valeurs de *gamma* (0.9, 0.7, 0.3).

3. Dans la suite de ce travail, nous allons utiliser le filtre de *Deriche* pour lisser un signal en temps réel. De même que dans l'exemple du filtre RIF, nous allons utiliser les E/S du SHARC pour l'acquisition et la restitution du signal analogique. Pour ce faire, nous avons besoin d'un générateur de signaux et d'un oscilloscope. Il faut connecter la sortie analogique du générateur (régler à ~1 volt) sur l'entrée analogique du kit (**J4, Audio In**) et la sortie TTL sur la broche **DAI_P1** (interruption **IRQ_P1**). Le résultat est à visualiser en connectant l'oscilloscope sur la sortie analogique du kit (**J6, HP Out**).

➤ Créer un projet appelé : **Der1D**

- Ajouter au projet les fichiers du dossier **Der1D** :
"*ADSP-21262.LDF*", "*init1835viaSPI.c*", "*initDAI.c*" et "*initSPORT.c*"
- Vérifier que l'exécutable obtenu effectue l'acquisition d'un signal analogique et le restitue sans aucun traitement.

Question : Que se passe-t-il lorsqu'on augmente la fréquence du signal d'entrée ?

4. **Mettre en place la gestion des 3 buffers** et vérifier l'acquisition et la sortie directe des échantillons. Cette étape doit faire appel aux 3 buffers tampons et à l'interruption **IRQ_P1**. Le résultat visuel de cette étape est identique à celui de l'étape 2 : acquisition + restitution du signal (sans traitement, mais circulation des échantillons par les 3 tampons).

5. **Traitement d'un signal carré**

Régler le générateur de signaux pour produire un signal carré de fréquence 100Hz. Utiliser la fonction **Lisseur1D** pour traiter les échantillons en fixant le coefficient **gamma** à 0.5

Questions :

- Que constatez-vous sur le signal de sortie (oscilloscope) ?
- Que se passe-t-il lorsqu'on fait varier la fréquence du signal d'entrée ?
- Donner la plage des fréquences pour lesquelles le filtrage est exploitable. Expliquer ?

6. **Changement du coefficient du filtre**

Ajouter une fonction pour faire varier le coefficient du filtre *gamma* par une simple action sur le bouton poussoir correspondant à **IRQ1**. Prenez un pas d'incréméntation de **0.125** à chaque appui.

Question : Que constatez-vous sur la sortie après plusieurs appuis (une dizaine) ?

7. **Fonction Lisseur1D en assembleur**

Ecrire en assembleur une version séquentielle (non optimisée) du filtre de *Deriche* "*Lisseur1D_ASM.asm*" et comparer les résultats obtenus à ceux de la fonction "*Lisseur1D.c*".

Remarque importante : Pour simplifier la réalisation de l'application, n'hésitez pas à commencer par coder uniquement la partie causale du filtre de *Deriche* (cellule de base ordre 1).

⇒ **Gros bonus : si version optimisée.**

8. **Mesure de la fréquence du signal d'entrée**

Ecrire en C une fonction qui mesure la fréquence du signal d'entrée et l'affiche sur les 8 LEDs en binaire naturel. N'oubliez pas de décharger le processeur de tout autre calcul.